

Comprehensive Development Cheat Sheet

Table of Contents

1. C# Fundamentals
2. Git Commands
3. Azure DevOps
4. SQL
5. Entity Framework
6. Dapper.NET
7. ADO.NET
8. Algorithm Analysis
9. Parallel Processing & Multi-Threading

C# Fundamentals

Data Types

```
// Value Types
int number = 42;
double price = 42.99;
bool isValid = true;
char letter = 'A';

// Reference Types
string text = "Hello";
object obj = new object();
dynamic dyn = "Dynamic type";

// Nullable Types
int? nullableNumber = null;
```

Collections

```
// Arrays
int[] numbers = new int[5];
string[] names = { "John", "Jane" };

// Lists
List<string> items = new List<string>();
items.Add("Item");
items.Remove("Item");

// Dictionary
Dictionary<string, int> ages = new Dictionary<string, int>();
ages["John"] = 25;

// HashSet
HashSet<int> uniqueNumbers = new HashSet<int>();
```

Object-Oriented Programming Concepts

```
// Class Definition
public class Person
{
    // Properties
    public string Name { get; set; }
    private int age;

    // Constructor
    public Person(string name)
    {
        Name = name;
    }

    // Method
    public virtual void Introduce()
    {
        Console.WriteLine($"Hello, I'm {Name}");
    }
}
```

```

}

// Inheritance
public class Student : Person
{
    public string StudentId { get; set; }

    public override void Introduce()
    {
        base.Introduce();
        Console.WriteLine($"My ID is {StudentId}");
    }
}

```

Troubleshooting Guide

1. Common Issues and Solutions

NullReferenceException

```

// Issue: Object not initialised
Student student = null;
string name = student.Name; // Throws NullReferenceException

// Solution: Check for null
if (student != null)
{
    string name = student.Name;
}
// Or utilise null conditional operator
string name = student?.Name;

```

Memory Management

```

// Issue: Resources not properly disposed
var stream = new FileStream("file.txt", FileMode.Open);
// Resource remains open

// Solution: Utilise using statement
using (var stream = new FileStream("file.txt", FileMode.Open))
{
    // Work with stream
} // Automatically disposed

```

Asynchronous Programming

```

// Issue: Deadlock in user interface applications
public void ButtonClick()
{
    var result = AsyncMethod().Result; // Potential deadlock

// Solution: Utilise async throughout
public async Task ButtonClickAsync()
{

```

```
var result = await AsyncMethod();  
}
```

2. Performance Optimisation

Database Operations

```
// Issue: Multiple database queries  
foreach (var id in ids)  
{  
    var user = await context.Users.FindAsync(id); // Multiple queries  
}  
  
// Solution: Single query with includes  
var users = await context.Users  
    .Where(u => ids.Contains(u.Id))  
    .ToListAsync();
```

Memory Usage

```
// Issue: Large object allocation  
var largeList = new List<string>(50000000); // Large initial capacity  
  
// Solution: Appropriate sizing  
var rightSizedList = new List<string>(expectedSize);
```

3. Debugging Strategies

Visual Studio Debugging

```
// Utilise conditional breakpoints  
// Right-click breakpoint -> Conditions  
if (count > 100) // Breakpoint condition  
{  
    Debug.WriteLine($"Count exceeded: {count}");  
}  
  
// Output window logging  
Debug.WriteLine("Debugging information");  
Trace.WriteLine("Trace information");
```

Exception Handling

```
try  
{  
    // Potentially problematic code  
}  
catch (Exception ex)  
{  
    // Log detailed information  
    Logger.LogError($"Error details: {ex.Message}");  
    Logger.LogError($"Stack trace: {ex.StackTrace}");  
}
```

Git Commands

Basic Commands

```
# Initialise repository
git init

# Clone repository
git clone <url>

# Add files to staging
git add <file>
git add . # all files

# Commit changes
git commit -m "message"

# Push/Pull
git push origin main
git pull origin main
```

Troubleshooting Git Issues

1. Merge Conflicts

```
# Issue: Merge conflict detected
git merge feature-branch
# CONFLICT detected

# Solution:
# 1. Open conflicted files
# 2. Resolve conflicts manually
# 3. Finalise:
git add .
git commit -m "Resolved merge conflicts"
```

2. Branch Management

```
# Issue: Detached HEAD state
# Solution:
git checkout main # Return to main branch
# Or create new branch from current state:
git checkout -b new-branch

# Issue: Wrong branch changes
# Solution: Stash changes
git stash
git checkout correct-branch
git stash pop
```

Azure DevOps

Azure Boards

```
# Work Item Types
- Epic
- Feature
- User Story
- Task
- Bug

# States
- New
- Active
- Resolved
- Closed
```

Troubleshooting Azure DevOps

1. Pipeline Issues

```
# Issue: Build pipeline failures
# Common solutions:
- Check NuGet package restoration
- Verify SDK version compatibility
- Ensure test dependencies are met

# Example pipeline fix:
trigger:
  branches:
    include:
      - main
      - develop

pool:
  vmImage: 'windows-latest'

steps:
- task: NuGetToolInstaller@1
  inputs:
    versionSpec: '5.x'
```

2. Release Management

```
# Issue: Failed deployments
# Checklist:
- Verify service connections
- Check environment permissions
- Validate variable groups
- Confirm deployment pool availability

# Example release fix:
steps:
- task: IISWebAppDeploymentOnMachineGroup@0
  inputs:
```

```
WebSiteName: "${WebsiteName}"  
VirtualApplication: "${VirtualApplication}"  
Package: "${System.DefaultWorkingDirectory}/**/*.zip"
```

SQL

Query Optimisation

```
-- Efficient SELECT queries
SELECT u.Name, o.OrderDate
FROM Users u
INNER JOIN Orders o ON u.ID = o.UserID
WHERE u.IsActive = 1
      AND o.OrderDate >= DATEADD(month, -1, GETDATE())
```

Troubleshooting SQL

1. Performance Issues

```
-- Issue: Slow query performance
-- Solution: Analyse execution plan
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO

-- Create appropriate indexes
CREATE NONCLUSTERED INDEX IX_Users_Email
ON Users(Email)
INCLUDE (Name, LastLoginDate);

-- Monitor lock timeouts
SELECT * FROM sys.dm_tran_locks
WHERE request_session_id > 50;
```

2. Data Integrity

```
-- Issue: Data inconsistency
-- Solution: Implement constraints
ALTER TABLE Orders
ADD CONSTRAINT FK_Orders_Users
FOREIGN KEY (UserID) REFERENCES Users(ID);

-- Validate existing data
SELECT *
FROM Orders o
LEFT JOIN Users u ON o.UserID = u.ID
WHERE u.ID IS NULL;
```

Entity Framework

Configuration

```
public class ApplicationDbContext : DbContext
{
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<User>()
            .HasIndex(u => u.Email)
            .IsUnique();

        modelBuilder.Entity<Order>()
            .HasOne(o => o.User)
            .WithMany(u => u.Orders)
            .HasForeignKey(o => o.UserId);
    }
}
```

Troubleshooting Entity Framework

1. Query Performance

```
// Issue: N+1 query problem
// Poor performance:
var users = await context.Users.ToListAsync();
foreach (var user in users)
{
    // Separate query for each user
    var orders = await context.Orders
        .Where(o => o.UserId == user.Id)
        .ToListAsync();
}

// Solution: Proper Include
var users = await context.Users
    .Include(u => u.Orders)
    .ToListAsync();
```

2. Migration Issues

```
// Issue: Failed migrations
// Solution: Reset migrations

// 1. Remove existing migration
public override void Down()
{
    // Reverse problematic changes
    DropTable("dbo.Users");
}

// 2. Add new migration
Add-Migration FixUserTable
Update-Database
```

Parallel Processing & Multi-Threading

Asynchronous Operations

```
// Proper async/await usage
public async Task<IEnumerable<User>> GetActiveUsersAsync()
{
    await using var context = new ApplicationDbContext();
    return await context.Users
        .Where(u => u.IsActive)
        .ToListAsync();
}
```

Troubleshooting Parallel Processing

1. Thread Synchronisation

```
// Issue: Race conditions
private int _counter;

// Problematic:
Parallel.ForEach(items, item =>
{
    _counter++; // Race condition
});

// Solution: Thread-safe operations
private int _counter;
Parallel.ForEach(items, item =>
{
    Interlocked.Increment(ref _counter);
});
```

2. Task Management

```
// Issue: Unhandled exceptions in tasks
// Solution: Proper error handling
public async Task ProcessItemsAsync(IEnumerable<Item> items)
{
    try
    {
        var tasks = items.Select(ProcessItemAsync);
        await Task.WhenAll(tasks);
    }
    catch (Exception ex)
    {
        Logger.LogError($"Processing failed: {ex.Message}");
        throw;
    }
}

// Proper cancellation
public async Task ProcessWithCancellationAsync(
    CancellationToken token)
{
}
```

```
while (!token.IsCancellationRequested)
{
    await Task.Delay(100, token);
    token.ThrowIfCancellationRequested();
}
```

Best Practices Summary

1. General Development

- Utilise proper exception handling
- Implement logging throughout
- Follow coding standards
- Write comprehensive unit tests

2. Database Operations

- Use parameterised queries
- Implement proper indexing
- Monitor query performance
- Maintain data integrity

3. Security Considerations

- Implement proper authentication
- Use secure communication
- Validate all inputs
- Follow security best practices

4. Performance Optimisation

- Profile code regularly
- Monitor resource usage
- Optimise database queries
- Implement caching where appropriate

5. Maintenance

- Document code thoroughly
- Maintain version control
- Perform regular updates
- Monitor system health